

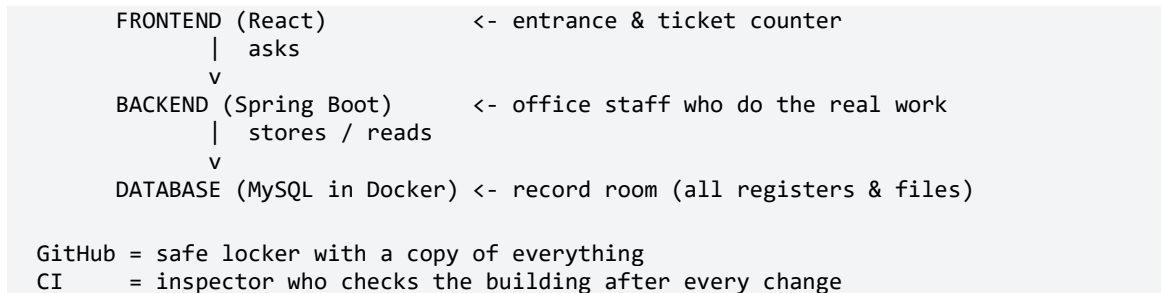
EventHub - Phase 1 Explained

"Building the house before living in it"

Madhavan Suresh | 28 September 2026 | Status: Steps 1-8 done, Step 9 pending

The big picture

Think of EventHub as **building a new college auditorium**. Phase 1 does not hold any events yet. It builds the **foundation, walls, electricity and water connection**, and checks that the lights switch on.



Step 1 - Tools: buying the construction equipment

Tool	Real-life role
Java 26 (JDK)	The language the office staff (backend) speak
IntelliJ IDEA	Workshop where you write backend code
VS Code	Workshop for frontend code
Node.js / npm	Runs React and downloads its parts (like a hardware shop)
Git	A diary that remembers every change you make
Docker	Ready-made "tiffin boxes" for MySQL and Mailpit
Postman	A "test customer" who asks the backend questions

Lesson learned (28 Sep): always open IntelliJ **yourself from the Start menu**. When Claude opened it, it left a broken lock file (.port) and IntelliJ could not start. Fix: rename the cache folder %LOCALAPPDATA%\JetBrains\IdeaIC2025.2 and open IntelliJ again.

Step 2 - GitHub: a locker in the bank

The code lives on your laptop **and** a copy is on GitHub (github.com/madhavansuresh000-sys/eventhub). If the laptop breaks, the code is safe. Every change is saved as a **commit**, like a diary entry:

```
Phase 1 step 4: React frontend ...
Phase 1 step 5: docker-compose ...
Phase 1 step 6: connect to MySQL ...
```

Step 3 - Spring Boot backend: hiring the office staff

The backend is **the brain**. Later it will book seats, stop overbooking and send emails. Right now it has one small job: answering "Are you alive?" (Step 7).

IntelliJ was set up properly: **SDK = JDK 26**, **language level = 25** (matches pom.xml), **Lombok plugin** installed, **compiler output = backend\out**.

Step 4 - React frontend: the entrance and ticket counter

This is what students **see and click**, like the BookMyShow website.

Part	Simple meaning
Vite	The engine. Save a file and the page updates instantly (hot reload).
Tailwind CSS	Ready-made paint and design: bg-green-100 = light green box
React Router	Different rooms: "/" = Home, "/about" = About
Redux Toolkit	Shared memory of the app, e.g. who is logged in (used later)
Axios	The messenger that carries questions to the backend

Step 5 - Docker: MySQL and Mailpit in tiffin boxes

Instead of installing MySQL on Windows (messy), **one command** starts everything:

```
docker compose up -d
|- eventhub-mysql    -> the database      (port 3306)
|- eventhub-mailpit  -> a fake email inbox (http://localhost:8025)
```

- **Mailpit:** when EventHub sends "Your ticket is booked", the email lands here, not in a real student's inbox. Safe for testing.
- **Passwords** are kept in **.env**, which **never goes to GitHub**. Only the sample file **.env.example** goes to GitHub.
- The **mysql-data volume** is like a fridge: your data stays safe even when the container stops.

Step 6 - Connect Spring Boot to MySQL: the record room key

Remember the red error "**Failed to configure a DataSource**"? The staff did not know **where the record room was**. Now **application.yml** gives the address, username and password. It has **2 profiles**:

```
dev -> your laptop: MySQL in Docker, Mailpit for emails, reads passwords from .env
prod -> real server later: settings come from the server (secret environment variables)
```

Like a **practice ground for rehearsal** and **the real stadium for the match**, each with its own settings.

Step 7 - Health check + Swagger: switching on the lights

- **/api/health** -> the backend answers {"status": "UP"}
- React asks this question when the page opens and shows **Backend: UP** in green.

```
React --"Are you alive?"--> Spring Boot
<----- "UP" -----
Page shows: Backend: UP (green)
```

- **Swagger UI** (<http://localhost:8080/swagger-ui.html>) -> an automatic **menu card** of all your APIs.
- **Security:** every other URL answers **401 = "please log in first"**. Real login comes in Phase 5.
- **3 automatic tests** check all this every time the tests run.

HTTP status codes to remember:

Code	Meaning
200 OK	Success, here is the answer
201 Created	Saved successfully (e.g. ticket booked)
400 Bad Request	You sent wrong data
401 Unauthorized	Not logged in
404 Not Found	Wrong URL
500 Server Error	A bug crashed the backend

Step 8 - GitHub Actions CI: an inspector after every change

Every time you push code, GitHub **by itself**:

1. Starts a fresh computer
2. Builds the backend and runs the tests (with its own temporary MySQL)
3. Builds the frontend and checks the code style (lint)
4. Shows a **green tick** (all good) or a **red cross** (something broke)

Like a **building inspector** who checks the auditorium after every renovation, so nothing is secretly broken.

Step 9 - Paper sketches: the architect's drawing (your task)

Draw on paper, take a photo, and it goes into the repo:

- **4 pages:** Home (event list), Event details, Login, My bookings
- **Database boxes:** users, events, bookings, with lines showing how they connect

Why paper first? Drawing first means you **know what to build** in Phase 2, just like an architect draws before the workers build.

Phase 1 summary

#	Step	Status
1	Tools	Done
2	GitHub repository	Done
3	Spring Boot backend + IntelliJ setup	Done
4	React frontend	Done
5	Docker: MySQL + Mailpit	Done
6	Connect Spring Boot to MySQL	Done
7	Health API + Swagger	Done
8	GitHub Actions CI	Done (green)
9	Paper sketches	Pending - your turn

"Done when" checks: React shows "Backend: UP" - **passed**. docker compose up starts the database with one command - **passed**. Green CI tick on GitHub - **passed**.

How to run everything (daily routine)

1. Open **Docker Desktop** (Start menu) and wait for "Engine running". Containers start automatically; if not: `docker compose up -d`
2. Open **IntelliJ** (Start menu) and run **EventhubApplication** -> backend on localhost:8080
3. In the **frontend** folder run `npm run dev` -> open `http://localhost:5173`

One-line summary

The building is ready: the entrance (React) talks to the office (Spring Boot), the office has the record room key (MySQL), everything is locked safely (GitHub + .env), and an inspector checks every change (CI).

Quick quiz

1. Why do we keep passwords in .env and not on GitHub?
2. What does 401 mean?
3. What is Mailpit used for?